

PAYSIM FRAUD DETECTION

A Full Machine Learning Project Report

From Raw Data to Explainable Model

EDA · Feature Engineering · Class Imbalance · Modelling · Explainability

Prepared by: Touhid Hasan
MSc Data Science | University of Greenwich
github.com/touhid-hasan
2026

Executive Summary

This report documents a full end-to-end machine learning project built on the PaySim synthetic financial transaction dataset. Six point three million transactions, 743 hours of simulated mobile money activity, and 8,213 labelled fraud cases. The goal was not simply to train a model and report an accuracy number. The goal was to understand fraud deeply enough to explain it clearly to anyone who needs to act on it.

The project moves through five stages: exploratory data analysis to understand what fraud actually looks like in the data, feature engineering to convert those observations into model inputs, class imbalance handling to address the 0.13 percent fraud rate, model training and comparison across three approaches, and SHAP explainability to make the model transparent and auditable. Each stage built directly on the one before it.

6.3M Transactions Analysed <i>743 hours simulated</i>	8,213 Fraud Cases <i>0.13% of all transactions</i>	98.1% Fraud Recall <i>best model performance</i>
---	--	--

The single most important finding of the entire project came not from a model but from reading the data carefully. Ninety-eight point one percent of all fraudulent transactions drain the origin account to exactly zero. That one observation generated the most powerful feature in the dataset, and it was visible in a basic scatter plot before a single line of modelling code was written. It is a reminder that domain understanding almost always matters more than algorithmic sophistication.

The best fraud detector is not the most complex model. It is the analyst who reads the data carefully enough to find the signal hiding in plain sight and then builds a system that acts on it reliably at scale.

1. Introduction

1.1 The Problem

Financial fraud is one of the most consequential applications of machine learning in the real world. Unlike many prediction problems where an incorrect output carries modest consequences, a missed fraud case means real money stolen from real people. A false alarm means a legitimate customer blocked and inconvenienced. The stakes on both sides are genuine and asymmetric, and getting the balance right requires both technical precision and commercial judgement. This project was built to practise exactly that combination.

1.2 The Dataset

The dataset contains 6,362,620 transactions recorded across 743 hours of simulated activity. Each transaction belongs to one of five types: CASH_OUT, PAYMENT, TRANSFER, CASH_IN, and DEBIT. The fraud label is binary. Eight thousand two hundred and thirteen transactions are fraudulent. The remaining 6,354,407 are legitimate.

Field	Type	Description
step	Integer	Hour of simulation (1 to 743)
type	String	Transaction type — 5 categories
amount	Float	Transaction amount in local currency
oldbalanceOrg	Float	Origin balance before transaction
newbalanceOrig	Float	Origin balance after transaction
oldbalanceDest	Float	Destination balance before transaction
newbalanceDest	Float	Destination balance after transaction
isFraud	Integer	Fraud label — 1 fraud, 0 legitimate

1.3 Project Objectives

- Identify the behavioural patterns distinguishing fraudulent from legitimate transactions through exploratory analysis.
- Engineer features that encode those patterns as numeric signals a model can learn from directly.
- Address extreme class imbalance so the model cannot achieve high accuracy by ignoring fraud entirely.
- Train and compare three models of increasing complexity with appropriate evaluation metrics.
- Explain model decisions at both global and individual transaction level using SHAP values.
- Produce a portfolio-ready project with clean documented code and findings communicable to a non-technical audience.

2. Exploratory Data Analysis

The EDA section treated the data as a set of questions rather than a checklist of standard plots. Each question was chosen because its answer would either confirm or challenge an assumption about fraud behaviour, and those answers shaped every subsequent design decision in the project.

2.1 Which Transaction Types Contain Fraud?

The first and most structurally important finding: out of five transaction types, only two contain any fraud at all. TRANSFER has a fraud rate of 0.77 percent. CASH_OUT has a fraud rate of 0.18 percent. PAYMENT, CASH_IN, and DEBIT have zero fraud cases across 6.3 million transactions. Not a low rate. Not negligible. Literally zero.

Transaction Type	Total Count	Fraud Cases	Fraud Rate
CASH_OUT	2,237,500	4,116	0.18%
PAYMENT	2,151,495	0	0.00%
CASH_IN	1,399,284	0	0.00%
TRANSFER	532,909	4,097	0.77%
DEBIT	41,432	0	0.00%

This is not a subtle statistical pattern. It is a hard rule. No fraud has ever been recorded in PAYMENT, CASH_IN, or DEBIT in this dataset. This single finding generates one of the most powerful features in the model: a binary flag that eliminates 56.5 percent of all transactions from fraud consideration before a single parameter is estimated.

2.2 Amount Distribution

The fraud median transaction amount is 441,423 dollars which is approximately six times higher than the legitimate median of 73,981 dollars. Fraudsters are not targeting small accounts or making modest transfers. They are deliberately selecting high value targets to maximise the return per attack. The log scale histogram makes this separation visually clear the legitimate distribution is wide and spread across the full amount range, reflecting the natural variety of real financial behaviour. The fraud distribution is narrower and shifted firmly to the right, clustering around a specific high value band. This is not noise. It is a signature.

The overlap zone between roughly 22,000 and 1.2 million dollars is where the real classification challenge lives. Both distributions are present here, and amount alone cannot distinguish fraud from legitimate. This is why multiple features working together are essential.

2.3 Temporal Patterns

By engineering hour and day features from the raw step column and calculating the fraud rate per hour, a clear temporal signature emerged. Fraud peaks at 4 AM with a fraud rate of 0.337 percent that is 2.61 times higher than the dataset average of 0.129 percent. This is not because more transactions happen at 4 AM. Transaction volume is actually relatively low at that hour. The fraud rate is genuinely elevated relative to activity level, confirming the spike as a behavioural signal rather than a volume artefact.

The most natural explanation is automation. Human fraudsters do not set alarms for 4 AM. Scripted attacks do. The timing is deliberately chosen to exploit the window of minimum human monitoring, minimum transaction volume, and minimum automated alerting.

2.4 Balance Patterns - The Drain Signature

This is the most striking finding in the entire project. Eight thousand and fifty-three out of 8,213 fraud cases 98.1 percent that drains the origin account to exactly zero balance after the transaction. Not approximately zero. Not close to zero. Exactly zero. No matter how large the account was to begin with, after a fraudulent transaction it is empty.

Four charts together told the complete story. A scatter plot of old versus new balance showed legitimate transactions forming a diagonal, money moves but accounts survive. Fraud collapses to a flat line along the y-axis. A bar chart quantified the 98.1 percent figure directly. A scatter of amount versus old balance confirmed that stolen amounts track almost perfectly with available balance and fraudsters take everything, not a fraction. And a destination balance scatter revealed that stolen funds do not settle at the receiving account but immediately disappear onward, confirming layering behaviour.

98.1 percent of fraud transactions drain the origin account to exactly zero. This single observation, visible in the raw data before any model is trained, generates the most powerful feature in the entire project. It is the clearest example in this analysis of why domain understanding matters more than algorithmic complexity.

3. Feature Engineering

Feature engineering is where EDA findings are converted into the language of machine learning numeric inputs the model can learn from directly. Every feature built in this section was grounded in a specific observation from the EDA. None of it was guesswork. The discipline of linking each feature directly to an analytical finding is what separates feature engineering from arbitrary column transformation.

3.1 Features Created

Feature	Formula	EDA Motivation
hour	$(\text{step} \text{ minus } 1) \bmod 24$	Encodes 4 AM fraud peak from Q3
day	$\text{floor}((\text{step} \text{ minus } 1) / 24) + 1$	Captures day-level patterns
is_transfer_or_cashout	1 if TRANSFER or CASH_OUT	Encodes Q1 finding — 100% of fraud
zero_balance_after	1 if newbalanceOrig = 0	Encodes 98.1% drain signature
errorBalanceOrig	$\text{newbalanceOrig} + \text{amount} - \text{oldbalanceOrig}$	Accounting anomaly at origin
errorBalanceDest	$\text{oldbalanceDest} + \text{amount} - \text{newbalanceDest}$	Accounting anomaly at destination
amount_to_balance_ratio	$\text{amount} / (\text{oldbalanceOrig} + 1)$	Fraction of balance stolen
log_amount	$\log(1 + \text{amount})$	Compresses 0 to 92M dollar range
type_encoded	LabelEncoder on type column	Numeric encoding for tree models

3.2 The Most Important Feature

zero_balance_after is a single binary yes or no question, did this transaction completely drain the account? It correctly identifies 98.1 percent of fraud before the model sees any other data. In practical terms this means a simple rule based system using only this feature would catch nearly all fraud in the dataset. The reason to build a full machine learning model on top of it is the 1.9 percent of fraud it misses and the false positive rate it generates among the legitimate transactions that also happen to reach a zero balance. The model combines this powerful signal with the others to achieve both high recall and acceptable precision simultaneously.

3.3 The Balance Error Features

In any legitimate financial transaction the accounting identity should hold exactly. Money leaving an origin account should exactly equal the transaction amount. Money arriving at a destination should exactly equal the transaction amount. The errorBalanceOrig and errorBalanceDest features measure the deviation from these identities. In legitimate transactions the errors are near zero. In fraud they are large and inconsistent that is a direct fingerprint of the record manipulation that accompanies fraudulent transfers.

4. Handling Class Imbalance

4.1 The Problem

The dataset contains 8,213 fraud cases out of 6,362,620 total transactions, a fraud rate of 0.129 percent. This extreme imbalance creates a fundamental problem: a model that predicts legitimate for every single transaction achieves 99.87 percent accuracy while catching zero fraud. Accuracy, in this context, is not just misleading. It is actively dangerous, because it makes a completely useless model look excellent on paper.

A naive model that predicts legitimate every time achieves 99.87 percent accuracy and catches zero fraud. This is why accuracy is the wrong metric for fraud detection and why the choice of evaluation metric is as important as the choice of model.

4.2 Approaches Compared

Two approaches were compared. Class weighting tells the model to penalise missed fraud cases approximately 387 times more heavily than missed legitimate cases, without changing the underlying data. SMOTE generates new synthetic fraud cases by interpolating between existing ones until fraud represents 10 percent of the training set. Both were applied and compared on the same test set.

Approach	Mechanism	Key Trade-off
No handling	Raw imbalanced data fed to model	High accuracy, near-zero recall
Class weights	Fraud penalty 387x higher	Better recall, no data manipulation
SMOTE	Synthetic fraud cases generated	Higher recall, risk of synthetic noise

4.3 Why Recall Matters Most

In fraud detection, the two types of error carry very different costs. A false negative a missed fraud case means real financial loss, potential regulatory exposure, and reputational damage. A false positive that is a wrongly flagged legitimate transaction, means customer inconvenience and analyst review time. In most financial services contexts a missed fraud case costs hundreds or thousands of times more than a false alarm. This is why recall, not accuracy, is the primary metric that matters for this problem.

5. Model Training and Comparison

5.1 Models Trained

Three models were trained to represent a meaningful progression from simple to complex, with each model providing a baseline for the next.

Logistic Regression — Baseline

The simplest possible model. A linear classifier that draws a single decision boundary through the feature space. With `class_weight` set to `balanced` it assigns 387 times more penalty to missing a fraud case. Its value is as a baseline: if more complex models cannot significantly outperform it, something has gone wrong in the feature engineering or preprocessing. Its coefficients also provide direct interpretability to each feature's contribution to fraud probability is visible and auditable without any additional explainability tooling.

Random Forest — Ensemble

Two hundred decision trees, each trained on a random subset of features and data, combined by majority vote. Where logistic regression draws one straight line, Random Forest draws thousands of nonlinear boundaries simultaneously. It naturally handles the complex interactions between engineered features particularly the combination of `zero_balance_after`, `amount_to_balance_ratio`, and `errorBalanceOrig` that together characterise the drain and layer fraud pattern. It also produces feature importance scores that either validate or challenge the EDA findings.

Gradient Boosting — Sequential Trees

Where Random Forest builds trees independently, Gradient Boosting builds them sequentially. Each new tree focuses specifically on the transactions the previous trees got wrong. This error correction mechanism consistently outperforms Random Forest on structured tabular data. Note: XGBoost, which is the production standard implementation of gradient boosting for structured financial data, was unavailable in the project environment. Sklearn's `GradientBoostingClassifier` was used, which implements the same mathematical principle at the cost of slower training speed.

5.2 Results

Model	Precision	Recall	F1 Score	AUC-ROC
Logistic Regression	0.0371	0.981	0.0715	0.9958
Random Forest	1.00	0.9976	0.9988	0.9997
Gradient Boosting	0.8770	0.9982	0.9937	0.9998

The logistic regression result deserves explanation rather than dismissal. A recall of 98.1 percent is excellent, the model catches nearly all fraud. But a precision of 3.7 percent means that for every 100 transactions it flags, 96 are legitimate false alarms. This reflects the aggressive class weighting forcing the model to err maximally toward flagging anything suspicious. The very low F1 score is a mathematical consequence of the harmonic mean pulling strongly toward the lower value when precision and recall are this far apart.

Random Forest and Gradient Boosting both achieve substantially better F1 scores by finding a more natural balance between precision and recall. This balance is driven by the engineered features providing clearer decision boundaries than the raw data alone could offer.

5.3 Understanding the Trade-off

A recall of 98.1 percent at 3.7 percent precision is not a broken model. It is a model configured aggressively for maximum fraud capture. In a real deployment the decision threshold would be tuned to the specific cost structure of the business. If a missed fraud case costs 10,000 dollars and a false alarm costs 5 dollars in analyst time, the optimal threshold sits very differently from a scenario where false alarms cost 500 dollars in customer churn. Threshold tuning, finding the probability cutoff that minimises total cost that is the natural step after initial training, and it is ultimately a business decision informed by data rather than a purely technical one.

6. Model Explainability with SHAP

Training a model that catches fraud is only half the work. In financial services a model whose decisions cannot be explained is a model that cannot be trusted, audited, or defended in a regulatory context. SHAP provides the bridge between a trained model and a human-readable account of its decisions.

6.1 What SHAP Does

SHAP assigns each feature a contribution value for every individual prediction. A positive SHAP value means that feature pushed the prediction toward fraud. A negative value means it pushed toward legitimate. The magnitude tells you how strongly. For any given transaction you can say precisely which features drove the fraud flag and by how much. This is not just analytically satisfying. It is what a fraud analyst needs to decide whether to block a transaction, what a compliance officer needs to justify an automated decision to a regulator, and what a data scientist needs to diagnose unexpected model behaviour in production.

6.2 Global Insight via Summary Plot

The SHAP summary plot shows every feature on the y-axis with each dot representing one transaction, coloured by feature value and spread horizontally by impact on fraud probability. The pattern it reveals should confirm the EDA and feature importance findings: balance related features dominate the rankings, amount contributes meaningfully, transaction type and time of day add incremental signal, and raw demographic identifiers carry negligible predictive information.

6.3 Individual Transaction Explanation

For any single transaction the SHAP waterfall plot shows the model's reasoning as a chain of contributions starting from the base rate of fraud across all transactions, then adding or subtracting each feature's contribution in turn, arriving at the final predicted probability. This is the output that makes the model auditable and actionable at the level of individual decisions.

The ability to explain a specific fraud flag , 'this transaction was blocked because it completely emptied the account, used a TRANSFER type, and occurred at 4 AM', is what separates a trustworthy production system from a black box. SHAP makes that explanation automatic and consistent for every single prediction the model makes.

7. Key Findings and Business Implications

7.1 Summary of Findings

- **100 percent of fraud occurs in TRANSFER and CASH_OUT transaction types. A single binary feature eliminates 56.5 percent of all transactions from fraud suspicion before modelling begins.**
- **98.1 percent of fraudulent transactions drain the origin account to exactly zero. This near-universal behavioural signature is the most powerful single feature in the entire model.**
- **Fraud transactions have a median amount six times higher than legitimate transactions , 441,423 dollars versus 73,981 dollars , reflecting deliberate high-value targeting.**
- **Fraud peaks at 4 AM at 2.61 times the dataset average rate, consistent with automated scripted attacks timed to exploit the minimum monitoring window.**
- **Destination account balances frequently fail to reflect incoming fraud amounts, confirming layering behaviour , stolen funds are immediately retransferred to obscure their trail.**
- **Balance accounting errors are significantly larger in fraudulent transactions, leaving a quantifiable fingerprint of the record manipulation that accompanies fraud.**

7.2 Business Implications

The concentration of fraud in TRANSFER and CASH_OUT types suggests that enhanced monitoring specifically for these transaction types — lower automatic block thresholds, additional authentication requirements, or higher-frequency human review — would catch the large majority of fraud with minimal disruption to the 80 percent of legitimate transactions occurring in other types.

The 4 AM fraud peak suggests automated monitoring systems should apply heightened sensitivity during overnight hours, and that human review capacity should be available or queued for morning review for transactions flagged in this window.

The account drain signature is directly actionable as a real-time rule independent of any machine learning model: any TRANSFER or CASH_OUT transaction that would bring an account balance to exactly zero and involves a high amount should be flagged for review automatically. This single rule catches 98 percent of fraud and can be implemented as a deterministic system check before the probabilistic model is even consulted.

8. Next Steps

- **Threshold tuning**: find the decision probability cutoff that optimises cost-adjusted performance for a specific business context.
- **XGBoost integration** : replace GradientBoostingClassifier with XGBoost once available for faster training and native class imbalance handling.
- **Full SHAP analysis** : complete summary, waterfall, and dependence plots for the best-performing model.
- **Streamlit dashboard** : deploy an interactive fraud prediction interface with live SHAP explanation per prediction.
- **Network analysis** : model account relationships over time to detect coordinated fraud rings rather than individual transactions.
- **Real-time pipeline** : implement model as a low-latency scoring API for production transaction blocking decisions.

9. Technical Appendix

9.1 Tools and Libraries

Tool	Purpose
Python 3.10+	Primary analysis language
pandas / NumPy	Data manipulation and numerical computation
Matplotlib / Seaborn	Visualisation
scikit-learn	Models, metrics, preprocessing, SMOTE
imbalanced-learn	SMOTE oversampling
SHAP	Model explainability
Streamlit	Interactive dashboard
joblib	Model serialisation

9.2 Model Configuration

Parameter	Logistic Regression	Random Forest	Gradient Boosting
Imbalance method	class_weight=balanced	class_weight=balanced	SMOTE 10%
Estimators	—	200 trees	200 trees
Max depth	—	20	5
Learning rate	C=0.1	—	0.05
Random state	42	42	42

9.3 Repository Structure

Path	Contents
notebooks/01_eda.ipynb	Full exploratory data analysis
notebooks/02_feature_engineering.ipynb	Feature creation and validation
notebooks/03_class_imbalance.ipynb	SMOTE and class weight comparison
notebooks/04_modelling.ipynb	Model training and comparison
notebooks/05_shap.ipynb	SHAP explainability analysis
src/features.py	Feature engineering functions
src/train.py	Training pipeline
models/fraud_model.pkl	Saved best model
app/streamlit_app.py	Interactive prediction dashboard

Path	Contents
outputs/	All generated charts
requirements.txt	Python dependencies
README.md	Project overview and findings

— END OF REPORT —